

Computer Programming Lab:7

One-Dimensional Arrays in C++

Prepared by:

Name: Hikmat Ullah Jan

Department: Electrical Comm

Semester: 2nd

Section: B

Reg No: BF25WNELE0662



Objectives

The objective of this lab is to:

- Understand what a one-dimensional array is
- Learn when to use arrays
- Study declaration and initialization of arrays
- Access and manipulate array elements
- Implement basic programs using arrays

One-dimensional array

Array is a data structure in C++, which stores a fixed size sequential collection of elements of the same type. An array is used to store a collection of data. It is often useful to think of an array as a collection of variables of the same type. All arrays consist of contiguous memory locations. The lowest address corresponds to the first element and the highest address to the last element.

A **one-dimensional array** is a collection of elements of the same data type stored in **contiguous memory locations**.

- It stores multiple values using a **single variable name**
- Each element is accessed using an **index**
- Index starts from **0**

Example:

```
int arr[5]; // Array of 5 integers
```

Declaration:

```
array_name[row][column]
```

Initialization:

It is like a variable, an array can be initialized. To initialize an array, we provide initializing values which are enclosed within curly braces in the declaration and placed following an equals sign after the array name. Here is an example of initializing an integer array.

Example:

```
int age[4]={23,34,64,75};int matrix[2][3] = {  
    {1, 2, 3},  
    {4, 5, 6}}
```

Example:

```
#include  
  
using namespace std;  
  
int main() {  
    // declaration of array  
    int age[4];  
  
    // initialization of array elements  
    age[0] = 23;  
    age[1] = 34;  
    age[2] = 64;  
    age[3] = 75;  
  
    return 0;  
}
```

An array of 4 integers is created. Values are assigned to each variable in the array on line (8) through line (11).

Accessing Elements:

In any point of a program in which an array is visible, we can access the value of any of its elements individually as if it was a normal variable, thus being able to both read and modify its value.

Syntax:

```
arrayName[index];
```

Following the previous examples in which “age” had 4 elements and each of those elements was of type int, the name which we can use to refer to each element is the following:

Examples:

```
age[0]; // 1st element of array age[1];
```

```
// 2nd element of array age[2];
```

```
// 3rd element of array age[3];
```

```
// 4th element of array Note that array index start with 0 in C++.
```

For example, to store the value 75 in the third element of age , we could write the following statement:

for example, to store the value of the third element of age to a variable called a, we could write:

Example: `a = age[2];`

Copying Arrays

Suppose that after filling our 4 element array with values, we need to copy that array to another array of 4 integers ?

Try this: Example:

```
1 4 5 6
#include <iostream>
using namespace std;
int main() {
    // declaration of array
    int age[4];
    // declare first array
    int same_age[4];
    // declare second array
    int i = 0;
    // initialization of array elements
    age[0] = 23; age[1] = 34; age[2] = 64;
    age[3] = 75;
    for (; i < 4; i++) {
        same_age[i] = age[i]; }
    for (i = 0; i < 4; i++) {
        cout << same_age[i] << " "; }
    return 0; }
```

<code>#include <iostream></code>	<code>// Include library</code>
<code>using namespace std;</code>	<code>// Use namespace</code>
<code>int main() {</code>	<code>// Main function</code>
<code> char message[] = "HELLO";</code>	<code>// Declare and initialize character array</code>
<code> cout << "Characters are:\n";</code>	<code>// Display message</code>
<code> for(int i = 0; message[i] != '\0'; i++) {</code>	<code>// Loop until null character</code>
<code> cout << message[i] << endl;</code>	<code>// Print each character on new line</code>
<code> return 0;</code>	<code>// End program</code>
<code>}</code>	

Task 3: Copying Arrays

- Declare two integer arrays: "source" and "destination", each with a size of 5.
- Initialize the "source" array with some values.
- Use a loop to copy the elements of the "source" array into the "destination" array.
- Display the values of both arrays to verify the copy operation.

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
int source[5] = {1, 2, 3, 4, 5};
```

```
int destination[5];
```

```
for(int i = 0; i < 5; i++) {
```

```
destination[i] = source[i];
```

$$\}$$

```
cout << "Source Array:\n";
```

```
for(int i = 0; i < 5; i++) {
```

```
cout << source[i] << " ";
```

$$\}$$

```
cout << "\nDestination Array:\n";
```

```
for(int i = 0; i < 5; i++) {
```

```
cout << destination[i] << " ";
```

$$\}$$

```
return 0;
```

}

```
// Include library
```

```
// Use namespace
```

```
// Main function
```

```
// Initialize source array
```

```
// Declare destination array
```

```
// Loop for copying
```

```
// Copy each element
```

```
// Display source array
```

```
// Loop through source
```

```
// Print source elements
```

```
// Display destination array
```

```
// Loop through destination
```

```
// Print destination elements
```

```
// End program
```

Task 4: Arrays Sum

- Write a C++ program that prompts the user to enter 5 numbers.
- Store the numbers in an integer array.
- Calculate the sum of all the numbers in the array.
- Display the calculated sum

```
#include <iostream>
using namespace std;
int main() {
    int arr[5];
    int sum = 0;
    cout << "Enter 5 numbers:\n";
    for(int i = 0; i < 5; i++) {
        cin >> arr[i];
    }
    for(int i = 0; i < 5; i++) {
        sum += arr[i];
    }
    cout << "Sum = " << sum;
    return 0;
}
```

```
// Include library
// Use namespace
// Main function
// Declare array
// Variable to store sum
// Prompt user
// Loop for input
// Store input in array

// Loop for sum
// Add each element to sum

// Display result
// End program
```